BioTerm-Bench v1: A Harbor-Compatible Benchmark for Tracking Open-Model Progress on Terminal Bioinformatics Tasks

Petr Simecek* Vojtech Bystry

Central European Institute of Technology (CEITEC), Masaryk University, Czechia

August 13, 2026

Abstract

Can a language model do your routine command-line bioinformatics unaided? For a surprising share of everyday work, the answer is already yes—and it is small, openly licensed models running on ordinary hardware that get there, not only frontier systems behind an API. BioTerm-Bench measures this directly: 32 self-contained tasks in which an agent is given real files, a container with the usual tools, and an exact specification of the output it must produce, and is graded by comparing that output against a known-correct answer. Evaluation is fully deterministic: no model judges another model's output, and grading reduces to comparison against a stored correct artifact. Examining agents on a basic set of 25 tasks, we found the benchmark already saturated—the strongest small open models solve essentially everything, one reaching 0.969 mean reward and answering all 25 correctly on its first attempt—so we added a frozen extension of harder multi-stage workflows. Across 14 models on two independently reported infrastructures, that extension separates models over a wide range (mean reward 0.433–0.875) while remaining unsaturated. Two findings matter most for practice. First, difficulty lives in *reliability*, not in what a model can eventually do. What matters for unattended analysis is that an agent succeeds almost every time, so alongside mean reward we report the fraction of tasks a model solves on *all four* attempts—a far more demanding criterion than solving a task at least once, and the one that corresponds to trusting a result without re-running it. Allowing four attempts and keeping the best hides most of the gap: on the extension all five models of one cohort solve every task under best-of-four and are therefore indistinguishable, yet under the all-four criterion they span two to seven tasks out of seven. Second, when these agents fail, the single most common outcome is not a crash or an unreadable file but a well-formed artifact containing wrong numbers (43.6% of 319 scored failures)—the failure mode least likely to be noticed downstream. The tasks, containers, graders, and analysis code are released freely so that a single laboratory can rerun the whole evaluation locally, including on infrastructure where sensitive genomic data must never leave the institution.

1 Introduction

A great deal of bioinformatics is not discovery. It is converting a GTF to a BED, checking that a BAM is coordinate-sorted, pulling allele frequencies out of a VCF, normalising counts to TPM—work that is precisely specified, tedious, and easy to get subtly wrong. This is exactly the shape of work where large language model (LLM) agents have started to be genuinely helpful, and the practical question for a laboratory is which parts can be delegated and how often the result will be right.

*Corresponding author: simecek@mail.muni.cz

Answering that question requires a benchmark with three properties that existing ones do not combine. It must run the tools rather than describe them, because an agent that writes a plausible `bcftools` invocation has not produced a filtered VCF. It must grade deterministically, because an LLM judge drifts between versions and cannot be audited by the person relying on it. And it should run on a laptop or an institutional node, so that the evaluation is cheap to repeat and so that it remains available for the subset of work where data cannot leave the institution.

There are large parts of bioinformatics without such constraints. Model-organism genetics, public reference data, and most method development can be sent anywhere, and for that work the case for local execution is ordinary rather than regulatory: cost, reproducibility, and not depending on an API that may be retired. But a substantial part of the field does carry the constraint, and it is the part where mistakes are most expensive: human genomic data under controlled access generally cannot be sent to unauthorised public generative-AI services [8]. For that work the realistic deployment is federated, with the agent running where the data already sits and orchestrating ordinary command-line tools on a node that exports nothing. Designing for the stricter case costs the easier one nothing, so three choices follow. Tasks are sized for smaller, locally runnable open models rather than only frontier APIs. Grading is fully deterministic, so a node can verify agent output automatically without calling out to anything. Reference data, tools, and graders are containerised, so the same evaluation reproduces on a laptop, a cluster node, or a partner institution's hardware.

Running that evaluation across 14 models produced a result we did not expect to state so bluntly: the 25-task core is largely solved. Two models answer all 25 tasks correctly on every one of four attempts. That is genuinely good news for practitioners—routine terminal work is now largely automatable by models a laboratory can host itself—and it is bad news for the benchmark, because a scoreboard where several entrants are perfect no longer ranks anything. So we did what benchmark maintainers must eventually do and built a harder pack, selecting eight multi-stage tasks before running any evaluation model, one of which we later retired, in the spirit of successively harder ARC-AGI generations.

What that harder pack revealed is the part most relevant to anyone deciding whether to hand work to an agent. The extension is not harder because models cannot eventually solve it; given four attempts, several solve nearly all of it. It is harder because models solve it *unreliably*, and reliability is what a single-shot user actually consumes. We therefore report mean reward—the fraction of individual attempts that come out right—as the primary number throughout, and treat best-of-four as a diagnostic rather than a score.

2 Related work, and the principles BioTerm-Bench stands on

BioTerm-Bench follows the executable-task tradition established by Terminal-Bench, which evaluates agents on hard, realistic tasks in genuine command-line environments and grades them with comprehensive tests [6]. We adopt that model directly: our tasks run under Harbor, the Terminal-Bench harness, so the suite is a domain-specific pack within an existing ecosystem rather than a new framework [4]. The contribution is the domain instantiation—bioinformatics file formats, tools, and pipelines—together with three properties that, to our knowledge, no existing biological benchmark combines.

Executable artifacts rather than code. BioCoder evaluates function-level bioinformatics code generation against unit tests [11], and ScienceAgentBench asks for a self-contained program derived from a published analysis [1]. Both are careful tests of code generation. BioTerm-Bench instead

supplies input files and a tool stack and holds the agent responsible for the final artifact, which shifts the difficulty toward format edge cases, coordinate conventions, and header requirements—the errors that dominate practical work.

Deterministic grading. BixBench evaluates open-ended exploration of biological datasets [7], BLADE compares an agent's analysis against expert-validated alternatives [3], and BioAgent Bench grades end-to-end workflow outputs using an LLM judge [2]. Model-based grading is appropriate for open-ended questions, where no single output is uniquely correct. It also introduces a dependency that changes as the judge model changes. Because every BioTerm-Bench task states its output contract precisely, grading reduces to comparison against a stored answer, which keeps scores comparable across time and auditable by the user.

Local execution. BioAgent Bench shares our motivation regarding sensitive data but is built around cloud infrastructure [2]. Closest in deployment assumptions is a recent evaluation of open-weight agents on longitudinal cohort-data preparation, motivated by governance rules that prohibit cloud transmission [9]; that work targets data cleaning rather than bioinformatics file and tool pipelines. Our fixtures are small enough to run on a laptop, and reference data, tools, and graders are containerised.

Benchmarks addressing scientific judgement are complementary. BioDSA-1K evaluates hypothesis validation across biomedical studies [12], and GeneBench-Pro targets multistage analyses in which a plausible wrong analytical choice changes the conclusion [5]. Those benchmarks ask whether an agent reaches a defensible scientific conclusion. BioTerm-Bench addresses the prior question of whether it can reliably produce a specified artifact, which is a weaker requirement but one that can be verified exactly.

3 Benchmark and analysis protocol

The suite contains 32 deterministic tasks in two parts: a 25-task core and a seven-task extension of harder multi-stage workflows added once the core began to saturate. Every task supplies a bounded local dataset, a container with the usual bioinformatics stack, an instruction that fixes the output contract exactly, and a verifier that returns a binary reward. Table 1 summarises what the tasks cover.

Two terms recur. An *agent* is the loop wrapped around a language model that lets it act rather than answer: it issues a shell command, reads what came back, decides the next command, and stops when it judges the task done. A *harness* is the software that runs that loop reproducibly—starting the container, enforcing timeouts, capturing the transcript, and invoking the grader. We use Harbor as the harness and hold the Terminus-2 agent fixed across all model comparisons, so that differences in score reflect the model rather than the scaffolding [4]. Throughout, a *trial* is one attempt by one model at one task, *reward* is the binary verifier outcome of a trial, and k is the number of attempts per task. pass@1 is the fraction of tasks solved on the first scheduled attempt, and pass@k the fraction solved on at least one of k attempts.

A worked example of one task—its fixture, exact output contract, and grader—is given in Appendix B.

Fixtures are reproducible by construction rather than novel by construction. Each is rebuilt by a script in the repository, either as a deterministic reduction of reference data shipped inside the image or from a fixed random seed, and the regeneration commands are recorded so a reviewer can reconstruct any fixture. This supports reproduction and auditing, but it is not a contamination

Table 1: What the suite covers. Core tasks are single operations on one or two files; extension tasks chain several stages, so an error early in the chain is not recoverable later.

Category	Tasks	Examples
alignment	7	BAM depth histograms, duplicate marking, soft-clipping statistics, SAM/CRAM round-trips, splice-junction support
variant	7	allele frequencies, normalisation, splitting by type, trio Mendelian checks, diploid consensus, compound heterozygotes, multi-sample QC
metadata	6	GTF to BED, longest ORF, header renaming, six-frame codon counts, longest-transcript CDS
quality control	6	per-base quality, FastQC-style summaries, paired-read adapter and orphan handling, contamination mixtures
search	5	best BLAST hit, MAFFT alignment identity, k-mer Jaccard matrices, reciprocal-best-hit ortholog clusters
RNA-seq	4	TPM normalisation, biotype summaries, featureCounts-style matrices, differential-expression contrasts

guarantee: inputs derived from public reference annotation could in principle be memorized, even though the required output contract is task-specific and the graders demand exact artifacts.

The repeated-run setting used in the main analysis is the clean OpenRouter $k = 4$ sweep already stored in this repository: 25 tasks, 4 attempts per task, and therefore 100 trials per model. We analyze all job directories matching `jobs/full25-k4-open-*`.

The main complication is operational filtering. We do not merge protocol failures with ordinary benchmark failures. Instead, each completed job is classified from the raw trial artefacts using three rules:

1. unfinished jobs are excluded;
2. a trial is an error if it has non-null `exception_info` or lacks a scored verifier reward;
3. jobs with error rate above 20% are excluded as protocol failures.

The 20% threshold is a convention rather than a tuned optimum, and it is not load-bearing: eight of the nine jobs excluded on this rule have error rates of 0.97 or above and the ninth is 0.60, so any cut between roughly 0.25 and 0.55 gives the same partition. It also handles a subtle but important case visible in the stored artefacts: Harbor can record a verifier reward of 0.0 on a trial that also has an explicit timeout exception, and such a trial should still count as an operational error rather than as a valid scientific zero.

Metrics. Our headline number is *mean reward*: the fraction of all scored attempts that produced a correct artifact. It answers the question a practitioner faces—if I hand this model one task, how often is the result right?—and uses every attempt rather than collapsing four into a binary.

Because unattended analysis needs an agent that succeeds *almost every time*, we also report the *all-attempts* rate: the fraction of tasks a model solved on all four attempts. This is deliberately demanding, and it is the criterion that corresponds to trusting an output without re-running it.

Table 2: Included OpenRouter $k = 4$ runs on the 25-task core, ordered by mean reward. Pass denominators are explicit scored-task counts.

Model	mean reward	pass@1	all-scored	mixed	pass@4	Cost (USD)
Gemma 4 26B A4B	0.969	25/25	22/25	3/25	25/25	0.29
Qwen 3.5 9B	0.878	24/25	18/25	7/25	25/25	1.37
Gemma 4 31B	0.920	22/25	22/25	2/25	24/25	0.20
GPT-OSS 20B	0.520	11/25	6/25	17/25	23/25	0.08
Mistral Small 3.2 24B	0.380	8/22	5/25	11/25	16/25	2.22
Qwen 3 30B A3B	0.283	7/24	2/25	11/25	13/25	0.92
Qwen 3 14B	0.381	9/20	3/25	9/25	12/25	1.15
Qwen 3 8B	0.131	3/20	1/25	5/25	6/25	0.70

We report pass@1 as a secondary single-shot view, and pass@k only as a contrast, because counting a task as solved when any of four attempts succeeds presupposes an oracle that identifies which attempt was right—the benchmark has one, a working scientist usually does not. Pass proportions carry exact Clopper–Pearson intervals over explicit scored denominators; mean-reward intervals are bootstrapped over task-level means. Cost and wall-clock are retained as operational diagnostics, not speed claims, because the runs used shared hosted services.

Operational filtering. Provider failures are not benchmark failures. Each completed job is classified from raw trial artefacts: unfinished jobs are excluded, a trial is an error if it carries an exception or lacks a scored verifier reward, and jobs whose error rate exceeds 20% are excluded as protocol failures. The threshold is a convention rather than a tuned optimum and is not load-bearing here—eight of the nine jobs it excludes have error rates above 0.97 and the ninth is 0.60. The rule also catches a subtle case: Harbor can record a reward of 0.0 on a trial that also timed out, which is an operational error rather than a scientific zero.

4 Results

The original 25 tasks are saturated

Table 2 reports the clean repeated OpenRouter cohort on the core: 23 jobs, of which 8 are usable and 15 are excluded as unfinished or operationally broken. The top of the table is saturated. Gemma 4 26B A4B reaches 0.969 mean reward and solves all 25 tasks on its first attempt, with Qwen 3.5 9B at 0.878 and Gemma 4 31B at 0.920. With only 25 task units the exact 95% interval for a perfect score is still 0.863–1.000, so adjacent rows overlap heavily and the table stratifies models into bands rather than ranking them; a small-model panel whose leader misses three of 98 scored attempts is itself the saturation signal. Category rates are flat (pooled pass@4 of 0.75 for alignment, quality control, RNA-seq and search, 0.69 for metadata, 0.68 for variant), so the core behaves as a competence floor rather than a separating leaderboard.

Repeated attempts still carry information even here: GPT-OSS 20B scores 0.520 mean reward while solving 11 of 25 tasks on the first attempt and 23 of 25 within four, which is exactly the gap a best-of-n report would hide. Operational failures are common enough to matter—the excluded set includes several 100/100-error jobs, and a billed Mistral Small 3.1 24B run is not scored because all 100 trials carry `AgentTimeoutError`.

Table 3: Extension, e-INFRA cohort (August 13–14, 2026), all nine models scored. `kimi-k3` is scored from a re-run after a provider outage invalidated its first attempt; both receipts are retained.

Model	mean reward	pass@1	all-scored	mixed	pass@4
Deepseek V4 Flash Thinking	1.000	7/7	7/7	0/7	7/7
Deepseek V4 Flash	0.923	6/7	5/7	2/7	7/7
Glm 5.2	0.929	6/7	6/7	1/7	7/7
Kimi K3	0.964	6/7	6/7	1/7	7/7
Mistral Medium 3.5	0.926	6/7	6/7	1/7	7/7
Qwen3.5 Int4	0.964	6/7	6/7	1/7	7/7
Gemma4	0.786	5/7	5/7	2/7	7/7
Qwen3.5 122B	0.679	5/7	2/7	4/7	6/7
Gpt Oss 120B	0.423	3/6	1/7	3/7	4/7

Table 4: Extension, OpenRouter cohort. Cost is billed provider cost, reported only because this provider bills per token.

Model	mean reward	pass@1	all-scored	mixed	pass@4	Cost (USD)
GPT-5.6 Luna	1.000	7/7	7/7	0/7	7/7	0.13
Inkling Small	1.000	7/7	7/7	0/7	7/7	0.59
Muse Glimmer 30B	1.000	7/7	7/7	0/7	7/7	1.64
Qwen 3.6 35B A3B	0.821	6/7	4/7	3/7	7/7	0.72
Nemotron 3.5 Lightning	0.571	4/7	2/7	5/7	7/7	0.36

A harder extension, and whether it discriminates

We therefore froze a harder pack of eight multi-stage tasks, selected on three development models before any evaluation model ran, requiring each candidate to show at least one pass and one failure at a pooled development pass rate between 10% and 80%. The procedure is transparent but circular by construction—tasks were kept because they separated three particular models—so the evaluation panel is a held-out test of the selection rule itself. Appendix A holds the per-task receipt.

Two cohorts answer the question, run under the identical protocol and reported separately because provider and model set are confounded across them: nine models on e-INFRA (Table 3) and five on OpenRouter (Table 4).

The answer is a qualified yes. Across the fourteen scored models mean reward runs from 0.423 to 1.000 on e-INFRA and from 0.571 to 1.000 on OpenRouter, so the extension separates capability over a wide range. It nonetheless saturates at the top, and one retired task is the reason.

One task was retired, and it cost the subset its ceiling. `bam-splice-junction-support` was solved by 2 of the 14 scored models, with a per-task mean reward of 0.06 on e-INFRA and 0.05 on OpenRouter, and an item–total correlation of -0.94: both successes came from models in the lower half of their cohort. A task failed by the strongest models and passed once by a weaker one is not measuring the ability the rest of the suite measures, so we retired it, keeping its trials in the raw receipts and excluding them from scored aggregates through a recorded exclusion rather than by deleting data. Retiring it improved the subset's psychometrics and immediately exposed a second problem: on the remaining seven tasks four of the fourteen models reach a perfect 1.000 mean reward. The retired task was the only thing keeping the extension unsaturated at the top, and it

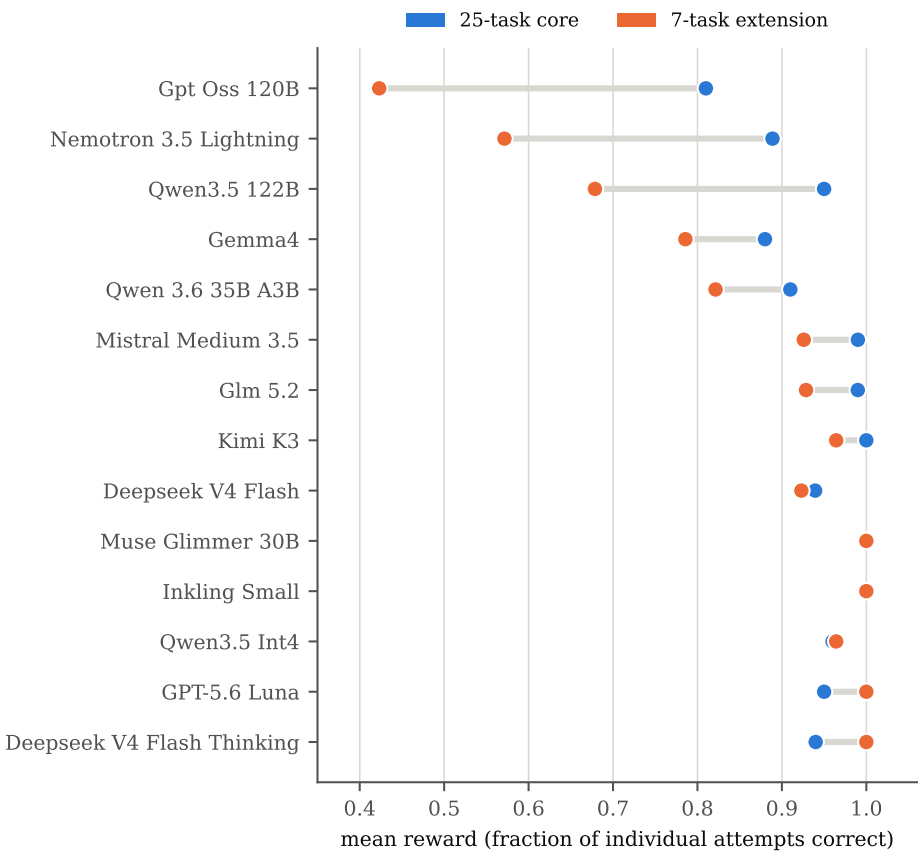


Figure 1: Every model evaluated on both parts of the suite. Each row is one model: blue is mean reward on the 25-task core, orange on the extension, and the grey bar is the drop. Models the core cannot separate—several sit at or near 1.000—fan out on the extension.

was doing so for the wrong reason—by being nearly impossible rather than by discriminating.

Difficulty moved into reliability, not solvability. Fourteen models completed *both* scopes on the same provider, so neither model set nor serving stack is confounded (Table 5, Figure 1). Averaged across them, moving from the core to the extension costs 3.0 times as much mean reward as pass@4 (0.087 against 0.029): markedly harder per attempt, and much less so in the limit.

The consequence for model comparison is stark in the OpenRouter cohort, where all five models solve every extension task within four attempts and are therefore *indistinguishable* under best-of-four. Under the all-attempts criterion the same five span two to seven tasks out of seven. A benchmark reported as best-of-n would call these models equivalent; a laboratory running each once would find one of them right twice as often as another.

An outage, and what it cost. Hosted evaluation is partly a systems benchmark. On its first attempt `kimi-k3` returned `InternalServerError` from the provider's inference proxy on 24 of its 32 trials, which the error-rate rule correctly excluded; a direct probe of the endpoint returned HTTP 500 on all four requests while the provider's public status page reported the model as online. Ten hours later the same probe succeeded and a re-run completed 32 of 32 trials with no errors. We keep both receipts: the failed run is an operational outcome, and the score comes from the

Table 5: Models with an included job in *both* scopes on the same provider. Comparing scopes across cohorts would confound task difficulty with model set and serving stack, so only these within-model pairs are admissible. Generated by `scripts/paired_scope_table.py`.

		mean reward		pass@1		pass@4	
Provider	Model	core	hard	core	hard	core	hard
e-INFRA	Deepseek V4 Flash	0.939	0.923	0.880	0.857	1.000	1.000
e-INFRA	Deepseek V4 Flash Thinking	0.940	1.000	0.960	1.000	1.000	1.000
e-INFRA	Gemma4	0.880	0.786	0.880	0.714	0.920	1.000
e-INFRA	Glm 5.2	0.990	0.929	0.960	0.857	1.000	1.000
e-INFRA	Gpt Oss 120B	0.810	0.423	0.800	0.500	0.960	0.571
e-INFRA	Kimi K3	1.000	0.964	1.000	0.857	1.000	1.000
e-INFRA	Mistral Medium 3.5	0.990	0.926	1.000	0.857	1.000	1.000
e-INFRA	Qwen3.5 122B	0.950	0.679	0.920	0.714	1.000	0.857
e-INFRA	Qwen3.5 Int4	0.960	0.964	0.960	0.857	1.000	1.000
OpenRouter	GPT-5.6 Luna	0.950	1.000	0.960	1.000	1.000	1.000
OpenRouter	Inkling Small	1.000	1.000	1.000	1.000	1.000	1.000
OpenRouter	Muse Glimmer 30B	1.000	1.000	1.000	1.000	1.000	1.000
OpenRouter	Nemotron 3.5 Lightning	0.889	0.571	0.917	0.571	1.000	1.000
OpenRouter	Qwen 3.6 35B A3B	0.910	0.821	0.960	0.857	0.960	1.000

clean re-run. A single evaluation window can misrepresent a model entirely, which is why error accounting belongs in the protocol rather than in a footnote.

Open weights and low price are independent properties. Because the OpenRouter provider bills per token, the extension sweep also prices each model, and the spread is larger than the capability spread (Figure 2). The cheapest run in the cohort is also tied for the best score, while the most expensive model costs roughly thirteen times more for an identical result. Price follows parameter count, serving efficiency, and provider margin, none of which follow from the licence. Openness buys the ability to run the weights inside your own boundary—a different and often more valuable property than a low per-token price, and the one that matters for the federated deployment this benchmark is designed around.

5 How agents fail

A binary verifier records *that* a run failed, not *how*, yet the two are not equally costly. Because every grader emits an explicit reason before returning zero, the distinction is measurable rather than anecdotal. We classified all 319 scored failures in the included core cohort from those reasons (Figure 3).

Failures divide into three kinds. In 68 cases (21.3%) the agent produced no output file at all—it crashed, ran out of time, or wrote to the wrong path. In 112 cases (35.1%) it produced a file that violated the declared structure: 38 had a wrong or missing header, 37 were unparseable JSON, 21 had the wrong number of columns, and 11 put a non-numeric value in a numeric field. The remaining 139 cases (43.6%) are the ones that matter most: the file parsed cleanly, satisfied every structural check, and disagreed with the correct answer. Within these, 97 were wrong numeric or string values in otherwise valid rows and 10 returned the wrong *set* of records—for example an empty BLAST hit list where five hits were expected.

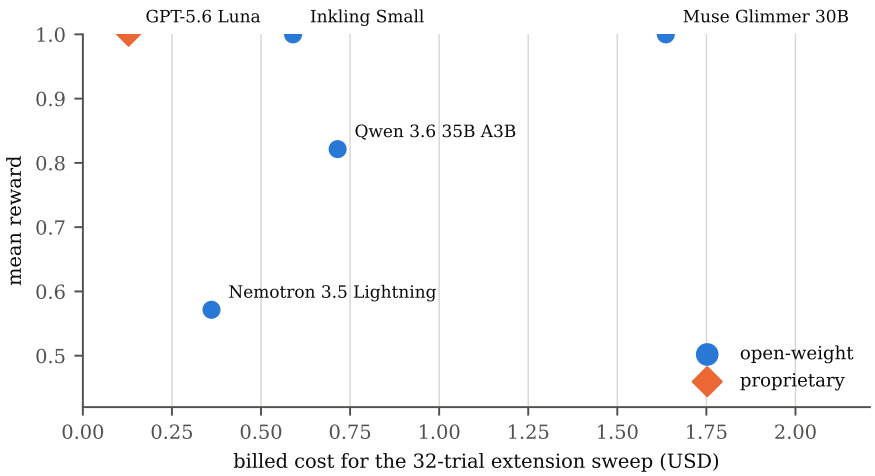


Figure 2: Billed cost against capability on the extension, OpenRouter cohort. The cheapest run is also tied for the best score, so price is a poor proxy for capability. Weight availability is declared in `configs/model_licences.tsv`.

The practical asymmetry is sharp. The first two kinds announce themselves: a downstream tool refuses to read a missing or malformed file, and the analyst notices immediately. The third kind does not. A BED file with correct syntax and wrong coordinates, or a flagstat summary with plausible but incorrect counts, propagates silently into whatever consumes it. That this quiet mode is the single largest category—and larger than the two loud ones taken separately—is the most important practical result in this paper.

It is also not confined to exotic tasks. The ten tasks contributing the most failures are ordinary operations: extracting the longest ORF, summarising FASTQ quality, checking trio Mendelian consistency, taking the best BLAST hit, converting GTF to BED. Header and column-count errors cluster where the output contract specifies an exact schema, which suggests that a meaningful share of failures is specification compliance rather than biology. That is encouraging for tooling—stricter output validation inside the agent loop would catch much of the loud 56%—but it does nothing for the quiet 44%, where only recomputing the answer reveals the error.

Which tasks are good measurements?

Retiring one item on an item–total correlation invites the obvious follow-up: what do the other items look like under the same lens? We fitted a two-parameter logistic item response model to every scored trial, treating each task as a test item,

$$P(\text{correct} \mid \text{model } m, \text{task } t) = \sigma(a_t(\theta_m - b_t)) \,,$$

with model ability θ_m, task difficulty b_t on the same scale, and discrimination a_t. The fit covers 14 models and 32 tasks; item parameters are each informed by every model's attempts, while ability estimates from fourteen respondents are correspondingly noisy and are reported only as an ordering.

Three results follow, and none of them is comfortable for the benchmark.

First, *every* task has negative difficulty on the fitted ability scale, the hardest—`blast-best-hit` at -0.79—still sitting well below the average model. This is the saturation result stated psychometrically rather than as a pass rate: the suite as a whole is calibrated below the population it is being used to measure.

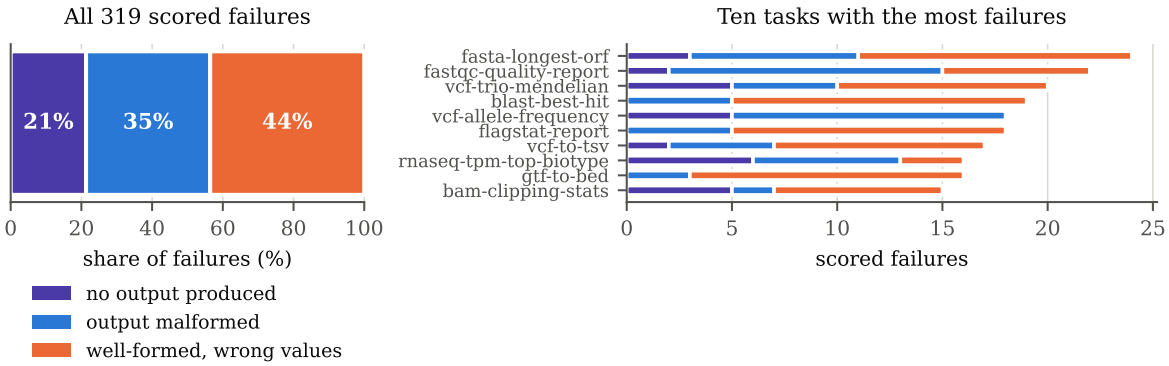


Figure 3: How runs fail. Left: all 319 scored failures in the included core cohort. Right: the ten tasks contributing the most failures, split the same way. Structurally valid output containing wrong values is the largest single category and is spread across ordinary tasks rather than concentrated in unusual ones.

Table 6: Item parameters for the ten most difficult tasks, from a 2PL fit over 14 models and 32 tasks. Difficulty is on the ability scale, so a negative value means the task sits below the average model. Discrimination is how sharply the item separates models around that point.

Task	Pass rate	Difficulty	Discrimination
`blast-best-hit`	0.75	-0.79	3.88
`vcf-multisample-qc`	0.79	-0.85	5.06
`vcf-trio-compound-het`	0.80	-0.86	7.93
`rnaseq-featurecounts-matrix`	0.84	-0.92	6.00
`vcf-trio-mendelian`	0.95	-1.25	13.65
`vcf-diploid-consensus`	0.78	-1.28	1.24
`gtf-to-bed`	0.89	-1.32	2.85
`bam-depth-histogram`	0.95	-1.35	5.30
`fastqc-quality-report`	0.93	-1.38	3.70
`fasta-longest-orf`	0.79	-1.40	1.17
9 tasks solved on every scored attempt – no measurement information			

Table 7: What it takes to self-host each model in the e-INFRA panel. Weight memory is total parameters at the served precision and excludes KV cache and runtime overhead. Tier A models are within reach of a workstation once quantised to four bits; tier E needs a multi-node cluster.

Model	Total (active)	Precision	Weights (GB)	Hardware needed
GPT-OSS-120B	120B (A5.1B)	`mxfp4`	60	B: one datacentre GPU
Gemma 4	31B (31B)	`bf16`	62	A: laptop or workstation
Qwen3.5 122B	122B (A10B)	`fp8`	122	B: one datacentre GPU
Mistral Medium 3.5	128B (128B)	`fp8`	128	B: one datacentre GPU
Qwen3.5 Int4	397B (A17B)	`int4`	198	C: 2–4 GPU server
DeepSeek V4 Flash	304B (A13B)	`fp8`	304	C: 2–4 GPU server
DeepSeek V4 Flash Thinking	304B (A13B)	`fp8`	304	C: 2–4 GPU server
GLM 5.2	756B	`fp8`	756	D: full 8-GPU node
Kimi K3	2.8T (A104B)	`fp8`	2800	E: multi-node cluster

Second, nine of the 32 tasks were solved on every scored attempt by every model. Their difficulty and discrimination are set by the prior rather than by data, because a task nobody fails carries no information about who is stronger. They remain useful as regression tests—a future model that fails `tpm-normalize` has broken something basic—but they contribute nothing to ranking and should not be counted as evidence that a model is capable.

Third, discrimination varies more than difficulty does, and the two are not aligned. `vcf-trio-mendelian` discriminates sharply ($a = 13.7$) despite a 0.95 pass rate, because the models that fail it are exactly the weaker ones. Meanwhile four tasks discriminate weakly at $a < 1$ despite non-trivial failure rates: `rnaseq-tpm-top-biotype`, `fasta-6frame-codon-counts`, `vcf-to-tsv`, and `kmer-contamination-mixture`. Failures on these are closer to noise than to signal—a strong model fails them about as often as a weak one—which makes them the next candidates for revision, and notably one of them is a member of the harder extension. Selecting future tasks on discrimination rather than on pass rate is the concrete methodological lesson: a task in the 10–80% band can still be a poor item, and our original selection rule could not tell the difference.

6 Discussion

What hardware do you actually need?

For a laboratory the practical question is not which model scores highest but which model it can run at all, and the two answers are further apart than one might expect. Table 7 places every model in the e-INFRA panel on a hardware ladder, using weight memory at the served precision—total parameters multiplied by bytes per parameter—as the sizing quantity. This is a floor rather than a deployment specification: the key–value cache for a 256K–1M context, activations, and runtime buffers add substantially on top, and a mixture-of-experts model needs its *whole* weight set resident even though only the active parameters run per token. As a sanity check on the arithmetic, the released MXFP4 weights of Kimi K3 are reported at roughly 1.4 TB, which is what 2.8T parameters at half a byte each predicts, and its vendor recommends 64 or more accelerators.

The ladder spans two orders of magnitude, from a 31B dense model that fits a laptop after 4-bit quantisation to a 2.8T mixture-of-experts model that fits no single machine. Capability follows it weakly and with a ceiling: across the eight scored models the rank correlation between weight memory and mean reward is $\rho = 0.77$ ($p = 0.027$), but the relationship saturates—the four models

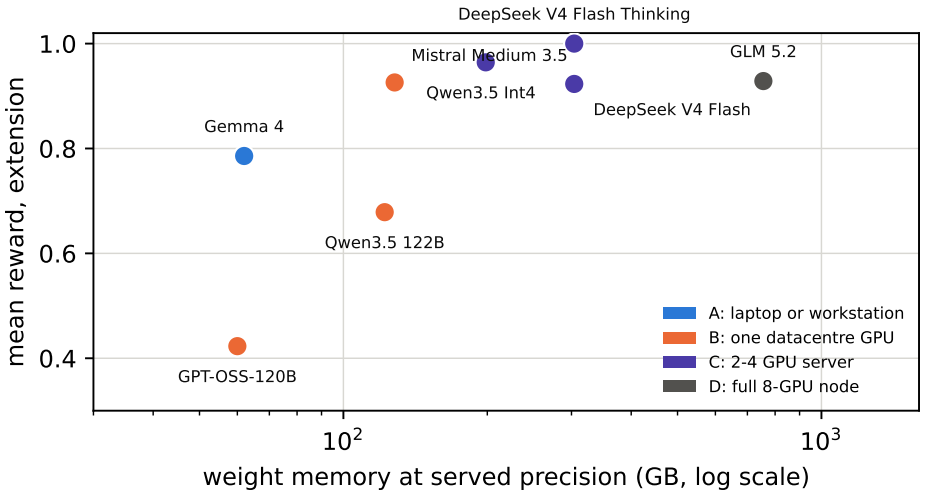


Figure 4: Capability against the hardware needed to host it. Colour marks the tier from Table 7. Larger models score better on average ($\rho = 0.77$, $p = 0.027$), but the relationship saturates above roughly 130 GB and admits large inversions: the 62 GB workstation-class model outscores a 122 GB model that needs a datacentre GPU.

at or above 130 GB score 0.800–0.875, so a move from a two-GPU server to an eight-GPU node buys nothing measurable—and admits large inversions, with 62 GB Gemma 4 outscoring 122 GB Qwen3.5 122B. Recency does not explain it either ($\rho = 0.53$, $p = 0.18$), and size and release date are themselves confounded in this panel ($\rho = 0.76$), so at $n = 8$ neither can substitute for running the benchmark on the model one intends to deploy.

Three deployment routes follow from this, and they map onto real institutional constraints. A laboratory with no privacy constraint and no GPU budget should use a commercial aggregator: the entire 8-task sweep cost between $0.15 and $1.94 per model, which is less than the electricity of running it locally, at the cost of sending prompts and data to third-party infrastructure. A laboratory working with controlled-access data and access to institutional hardware—the arrangement e-INFRA CZ provides for ELIXIR CZ members—can host a tier B or C model inside the administrative boundary and reach the top of the observed score range. A laboratory that must keep data on a single machine is restricted to tier A, and the encouraging finding is that this is no longer a severe restriction: a workstation-class model already solves the great majority of the 25-task core and roughly two thirds of the extension.

We did not measure tier A execution directly, and that is the main gap in this analysis. Our fixtures are deliberately sized for it, but confirming which models remain usable at 4-bit on consumer hardware—and how much quantisation costs in reliability rather than in benchmark score—requires its own study.

What to delegate today

The per-task results support a more specific recommendation than a single score. Format conversion and summarisation—GTF to BED, VCF to TSV, FASTA statistics, flagstat-style counting—are solved reliably by every model at the top of both cohorts, and are reasonable to delegate with spot-checking. Multi-stage reasoning over biological structure is not: joining CIGAR-derived splice junctions to annotations was solved by 2 of 13 models, and trio-aware and diploid-consensus variant

logic separates models sharply. Between these lie tasks that are solved but not dependably, where the model produces a correct answer most of the time and a plausible wrong one otherwise; these are exactly the cases where an unchecked result is most costly, and where a second run and a comparison is cheap insurance.

The general rule this suggests is that delegation is safe in proportion to how mechanically checkable the output is. An agent that writes a BED file can be checked by re-deriving a handful of rows; an agent that produces a filtered variant set with a subtle inheritance error cannot be checked without redoing the analysis.

Limitations and reproducibility

One operational deviation is recorded for completeness: a single core-25 job (`deepseek-v4-flash`) ran at four concurrent trials rather than two, because a ceiling change was applied to an already-running orchestrator that had loaded the previous value. Concurrency affects queueing and wall-clock, not task content or grading, so the job's scores are retained; its wall-clock is not comparable with the rest of the cohort, which is in any case not reported as a model-speed measurement.

This draft stops short of claiming final e-INFRA results that have not yet been run: the harder-task subset is frozen, but the e-INFRA cohort remains an extension in progress, so the quantitative claims here are limited to artefacts stored in the repository as of August 12, 2026.

Four limitations bound the analysis. The main table uses only the clean repeated OpenRouter cohort rather than the older $k = 1$ and $k = 3$ sweeps; 25 core tasks make every interval wide; cost and wall-clock are not comparable across shared hosted services; and holding the scaffold fixed at Terminus-2 controls a large variance source while bounding external validity, since a different scaffold can move scores substantially for the same model.

A binary verifier records that a task failed without recording how it failed, and the two failure families have very different practical costs. An absent or malformed artifact is cheap, because it announces itself: a downstream tool refuses to read it. A well-formed but incorrect artifact is expensive, because nothing flags it and the error propagates into whatever consumes it. Because our graders emit an explicit reason before returning zero, this split is measurable rather than speculative. Classifying all 319 scored failures in the included cohort by the verifier's own reason gives 68 (21.3%) missing outputs, 112 (35.1%) structurally invalid outputs—wrong header, wrong column count, unparseable JSON, non-numeric fields—and 139 (43.6%) outputs that parsed cleanly and disagreed with the gold answer. The quiet failure mode is thus the single largest category, and it is spread across ordinary tasks rather than concentrated in exotic ones: `blast-best-hit`, `fasta-longest-orf`, `flagstat-report`, `gtf-longest-transcript-cds`, and `gtf-to-bed` each contribute 13–14 such failures. For a reader weighing unsupervised delegation this is the most important number in the paper: when these models fail, the most common single outcome is a plausible file containing wrong numbers. The classifier is released as `scripts/summarize_failure_modes.py` with its per-trial output, so the taxonomy can be recomputed on any future cohort.

Reproducibility is built in: the repository contains the summarizer, the generated CSV and LaTeX tables, and explicit exclusion logic, so every number and figure here regenerates from raw receipts with one command. The three task images pin every bioinformatics dependency to an explicit version—`samtools` and `bcftools` 1.21, `minimap2` 2.28, `bedtools` 2.31.1—though they are published by tag rather than immutable digest, which is worth hardening before archival.

Availability. Tasks, images, runners, graders, the analysis scripts used for every table in this paper, and the frozen selection receipt are available at `https://github.com/ML-Bioinfo-CEITEC/`

`BioTerm-Bench`. Raw Harbor job directories are large and are retained locally; the derived per-model, per-task, and per-failure CSVs that back the tables are generated by `scripts/summarize_bioterm_v2.py` and `scripts/summarize_failure_modes.py`. This draft therefore provides a concrete baseline for the next iteration: land the harder tasks, run the e-INFRA cohort under the same repeated protocol, and keep protocol failures visible as first-class experimental outcomes rather than burying them in summary averages.

Future directions

Three extensions would sharpen what this benchmark can say.

The first is to close the deployment gap. We evaluated hosted inference on a commercial aggregator and on institutional hardware, but not the strictest case: weights executed on the analyst's own machine through a local runtime such as Ollama [10] or LM Studio. Our fixtures are sized for it, and the tier A result—a workstation-class model already solving most of the core—suggests the answer would be encouraging, but quantisation to four bits may cost reliability in ways a benchmark score hides.

The second is to look backwards rather than forwards. Every model here was released within roughly a year, which is why size and recency could not be separated. Evaluating models from the preceding two years—many no longer served by either provider, but still available as weights—would turn a snapshot into a trajectory and show how quickly routine bioinformatics became automatable.

The third is to keep the suite ahead of the field. Nine of 32 tasks are already solved on every attempt by every model, and retiring the one genuinely hard extension task left the remainder saturated at the top. The next pack should be selected on discrimination rather than pass rate, and should deliberately include tasks that current frontier models fail.

7 Conclusion

BioTerm-Bench is close to the point where its original 25-task core is no longer sufficient as the primary discriminator among current small open models. The existing OpenRouter artefacts already show partial saturation in mean reward, continued benefit from repeated attempts, and enough provider-path instability that operational filtering must be part of the benchmark protocol. The core keeps its value as a continuity score and as evidence of what is now routinely automatable, while the harder pack carries the discriminative load until it too saturates.

For a practitioner weighing whether an agent can absorb routine terminal work, the evidence here supports a narrow conclusion rather than a comfortable one. On tightly specified single-artifact tasks the strongest small open models are usually right, and repeated attempts recover much of what a single attempt misses. But the recovery requires someone who can tell a right answer from a wrong one, and the most common single failure mode is a well-formed file containing wrong numbers. The realistic role today is therefore an accelerator for someone who could have done the task themselves, not a replacement for that person.

Data availability

The task suite, container definitions, graders, runners, and analysis scripts are available at `https://github.com/ML-Bioinfo-CEITEC/BioTerm-Bench` under the Apache 2.0 licence. The derived per-model, per-task, and per-failure tables that back every figure and table in this paper are regenerated

from raw run receipts by `scripts/refresh_paper_tables.sh`. Raw Harbor job directories are large and are retained by the authors; they are available on request.

Conflict of interest

The authors declare no competing financial or non-financial interests.

Acknowledgements

This work was supported by [FUNDING BODY] under grant [GRANT NUMBER]. Computational resources were provided by the e-INFRA CZ project (ID:90254), supported by the Ministry of Education, Youth and Sports of the Czech Republic.

References

[1] Ziru Chen, Shijie Chen, Yuting Ning, Qianheng Zhang, Boshi Wang, Botao Yu, Yifei Li, Zeyi Liao, Chen Wei, Zitong Lu, Vishal Dey, Mingyi Xue, Frazier N. Baker, Benjamin Burns, Daniel Adu-Ampratwum, Xuhui Huang, Xia Ning, Song Gao, Yu Su, and Huan Sun. ScienceAgentBench: Toward rigorous assessment of language agents for data-driven scientific discovery. In *International Conference on Learning Representations*, 2025. URL `https://proceedings.iclr.cc/paper_files/paper/2025/hash/f12b4df26344f3be803c06b555252efe-Abstract-Conference.html`.

[2] Dionizije Fa, Marko Čuljak, Bruno Pandža, and Mateo Čupić. BioAgent Bench: An AI agent evaluation suite for bioinformatics. *arXiv preprint arXiv:2601.21800*, 2026. URL `https://arxiv.org/abs/2601.21800`.

[3] Ken Gu, Ruoxi Shang, Ruien Jiang, Keying Kuang, Richard-John Lin, Donghe Lyu, Yue Mao, Youran Pan, Teng Wu, Jiaqian Yu, Yikun Zhang, Tianmai M. Zhang, Lanyi Zhu, Mike A. Merrill, Jeffrey Heer, and Tim Althoff. BLADE: Benchmarking language model agents for data-driven science. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, 2024. arXiv:2408.09667.

[4] Harbor Framework Team. Harbor: A framework for evaluating and optimizing agents and models in container environments, January 2026. URL `https://github.com/harbor-framework/harbor`.

[5] Jeremiah H. Li, Suyash Shringarpure, Edmund Wong, and Andrew J. Ho. GeneBench-Pro: Evaluating multistage statistical reasoning in genomics, quantitative biology, and translational biomedicine. *bioRxiv*, 2026. doi: 10.64898/2026.06.29.735386.

[6] Mike A. Merrill, Alexander G. Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, et al. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces. *International Conference on Learning Representations*, 2026. URL `https://arxiv.org/abs/2601.11868`.

[7] Ludovico Mitchener, Jon M. Laurent, Benjamin Tenmann, Siddharth Narayanan, Geemi P. Wellawatte, Andrew White, Lorenzo Sani, and Samuel G. Rodriques. BixBench: A comprehensive benchmark for LLM-based agents in computational biology. *arXiv preprint arXiv:2503.00096*, 2025. URL `https://arxiv.org/abs/2503.00096`.

[8] National Institutes of Health. Protecting human genomic data when developing generative artificial intelligence tools and applications. Notice NOT-OD-25-081, 2025. URL `https://grants.nih.gov/grants/guide/notice-files/NOT-OD-25-081.html`.

[9] Mack Nixon, Liam Wright, Yevgeniya Kovalchuk, Alison Fang-Wei Wu, Martin Danka, Andy Boyd, and David Bann. Agentic coding without the cloud: Evaluating open-weight large language models on longitudinal data preparation tasks. *arXiv preprint arXiv:2607.21482*, 2026. URL `https://arxiv.org/abs/2607.21482`.

[10] Ollama. Ollama: Get up and running with large language models locally. `https://ollama.com`, 2026. Accessed 2026-08-14.

[11] Xiangru Tang, Bill Qian, Rick Gao, Jiakang Chen, Xinyun Chen, and Mark Gerstein. BioCoder: A benchmark for bioinformatics code generation with large language models. *Bioinformatics*, 40 (Supplement_1):i266–i276, 2024. doi: 10.1093/bioinformatics/btae230.

[12] Zifeng Wang, Benjamin Danek, and Jimeng Sun. BioDSA-1K: Benchmarking data science agents for biomedical research. *arXiv preprint arXiv:2505.16100*, 2025.

A Hard-subset selection receipt

Table 8 reproduces the frozen per-task development record that fixed the retained eight tasks before any evaluation model was run. It is included so the selection can be audited against the evaluation outcome in Section 4 without re-deriving it from the repository.

Table 8: Frozen retained hard-subset tasks from `notes/v2_task_selection.json`. Rates use only scored development attempts; errors are preserved separately and do not enter the rate denominator.

Task	Category	Passes / scored	Errors	Rate
`bam-splice-junction-support`	alignment	1 / 4	2	0.25
`vcf-diploid-consensus`	variant	1 / 6	0	0.17
`vcf-trio-compound-het`	variant	1 / 5	1	0.20
`vcf-multisample-qc`	qc	2 / 5	1	0.40
`rnaseq-featurecounts-matrix`	rnaseq	2 / 6	0	0.33
`rnaseq-deseq2-contrast`	rnaseq	2 / 5	1	0.40
`kmer-contamination-mixture`	qc	2 / 6	0	0.33
`ortholog-rbh-clusters`	search	4 / 6	0	0.67

B A worked example

At the atomic end of the ladder, `gtf-to-bed` gives the agent a 21 KB GTF file of chromosome 22 gene annotations inside a container carrying the usual HTS stack, and an instruction that fixes the output contract exactly: write `/app/output/exons.bed` as a BED6 TSV with one row per `exon` record, `start` converted to 0-based half-open coordinates, `name` taken from the `gene_id` attribute with quotes stripped, a literal `.` in the score column, rows sorted by chromosome, start, end, and name, and duplicate rows preserved rather than collapsed. The grader rejects any row that does not carry exactly six tab-separated columns and then compares the produced file against a stored gold file after normalization; the reward is 1 on an exact match and 0 otherwise. No model judges any output. Tasks at the hard end of the ladder keep the same contract discipline but require several coupled stages, such as joining CIGAR-derived splice junctions to GTF annotations.

C Audit tables for the core-25 cohort

These two tables are retained for auditing rather than for the argument in the main text: Table 9 records which jobs the error-rate rule removed and why, and Table 10 records residual task-level difficulty in the current core.

Table 9: Excluded OpenRouter jobs. These are not counted as scientific failures because the trial artefacts do not support normal benchmark interpretation under the error-rate policy.

Model	Reason	Error rate	Error trials	Verifier cov.
GLM 4.5 Air	protocol failure high error rate	1.00	100	0.00
Ministral 14B	protocol failure high error rate	1.00	100	0.00
Ministral 8B	protocol failure high error rate	1.00	100	0.00
Mistral Nemo	protocol failure high error rate	1.00	100	0.00
Mistral Small 2501	protocol failure high error rate	0.60	60	0.49
Mistral Small 2603	protocol failure high error rate	1.00	100	0.00
Mistral Small 3.1 24B	protocol failure high error rate	1.00	100	1.00
Qwen 2.5 7B	protocol failure high error rate	0.97	97	0.11
Qwen 3.6 Flash	protocol failure high error rate	1.00	100	0.00
Ministral 3B 2512	unfinished	0.02	2	0.03

Table 10: Hardest tasks in the current clean OpenRouter cohort. "Disc." is an item-discrimination statistic computed against the model-level pass@4 totals on the remaining tasks.

Task	Category	pass@4 models	Mean reward	Disc.
fastqc-quality-report	qc	3/8	0.208	0.73
blast-best-hit	search	3/8	0.240	0.69
fasta-longest-orf	metadata	4/8	0.250	0.91
vcf-trio-mendelian	variant	4/8	0.312	0.91
gtf-to-bed	metadata	4/8	0.375	0.91
vcf-to-tsv	metadata	4/8	0.417	0.62